Engineering Memory Copilot Documentation

SUMMARY

Engineering Memory Copilot is an AI-powered knowledge management platform that transforms engineering documents into persistent organizational memory using Cognee. Unlike traditional Retrieval-Augmented Generation (RAG) systems that retrieve isolated document chunks based on vector similarity, Engineering Memory Copilot organizes information into project-specific memories, enabling the AI to understand relationships between documents and provide context-aware responses.

PROJECTS

Engineering Memory Copilot | AI-powered knowledge management platform for engineering teams.

- Preserve engineering knowledge.
- Improve information retrieval.
- Accelerate developer onboarding.
- Connect related information across multiple documents.
- Build project-specific AI memory.
- Provide context-aware answers.
- Demonstrate Cognee's memory capabilities.
- Projects isolate knowledge into independent workspaces containing documents, memories, and chat history.
- Users can upload, view, and delete engineering documents (primarily PDF files).
- AI chat retrieves relevant memories before generating responses.
- Memory Explorer displays memories extracted by Cognee as structured organizational knowledge.
- Service-oriented backend includes Project Service, Document Service, Chat Service, and Cognee Service.
- Document processing workflow: upload document, store metadata, extract text, send to Cognee, create memory, searchable memory.
- Chat workflow: submit question, retrieve memories, combine context, LLM generates answer, return response.
- Advantages over traditional RAG include persistent memory, project-based knowledge, connected concepts, and context-aware responses.

TECHNICAL SKILLS

React
TypeScript
Tailwind CSS
FastAPI
Python
PostgreSQL
Cognee
OpenAI Compatible LLM

API ENDPOINTS

Projects: POST /projects, GET /projects, GET /projects/{id}, DELETE /projects/{id} Documents: POST /projects/{id}/documents, GET /projects/{id}/documents, DELETE /documents/{id} Memory: GET /memory/{project_id} Chat: POST /chat

HIGH-LEVEL ARCHITECTURE

Frontend → FastAPI Backend → Project Service → Document Service → Chat Service → Cognee Service → Cognee Memory → LLM → Response

DATABASE ENTITIES

Projects: id, name, description, created_at Documents: id, project_id, filename, path, upload_time Chats: id, project_id, question, answer, timestamp

DESIGN DECISIONS

Why Project-Based Isolation: keeps unrelated knowledge from mixing and improves accuracy. Why Service-Oriented Architecture: improves maintainability, testability, scalability, and readability. Why Cognee: enables persistent memory, knowledge relationships, context preservation, and organizational memory. Why PostgreSQL: reliability and transactional consistency for application metadata.

CURRENT LIMITATIONS

Single-user workflow, PDF documents, project-level isolation, manual document upload.

FUTURE IMPROVEMENTS / ROADMAP

Authentication, Role-Based Access Control (RBAC), multi-user collaboration, feedback-driven memory refinement, versioned memories, Slack integration, Confluence integration, GitHub integration, Jira integration, automatic document synchronization, hybrid graph + vector retrieval. Phase 1: Authentication, user management. Phase 2: Team collaboration. Phase 3: Enterprise integrations. Phase 4: Memory analytics. Phase 5: Autonomous knowledge updates.